where $\Psi_k(c_{I_k})$ is a function that only depends on the clique of random variables C_{I_k}.

By this factorization, if $p(c_{(i\to 1)}|d) > 0$ and $p(c_{(i\to 0)}|d) > 0$,

$$\ln \frac{p(c_{(i\to 1)}|d)}{p(c_{(i\to 0)}|d)} = \beta(c_{I_i}, d_{I_i})$$

where β is some function that only depends on cliques that involve C_i. In other words, $i \in I_i$ and $i' \in I_i$ if $C_{i'}$ and C_i are in the same clique in G_C.

Thus,

$$\ln \frac{p(c_{(i\to 1)}|d_{(j\to\diamond)})}{p(c_{(i\to 0)}|d_{(j\to\diamond)})} = \ln \frac{p(c_{(i\to 1)}|d_{(j\to c_j)})}{p(c_{(i\to 0)}|d_{(j\to c_j)})}$$

and,

$$\ln \frac{\hat{p}(c_{(i\to 1)}|d_{(j\to\diamond)})}{\hat{p}(c_{(i\to 0)}|d_{(j\to\diamond)})} = \ln \frac{\hat{p}(c_{(i\to 1)}|d_{(j\to c_j)})}{\hat{p}(c_{(i\to 0)}|d_{(j\to c_j)})}$$

Therefore,

$$\Big(g(c_{(i\to 1)}) - g(c_{(i\to 0)})\Big)^T f(d_{(j\to\diamond)}) = \Big(g(c_{(i\to 1)}) - g(c_{(i\to 0)})\Big)^T f(d_{(j\to c_j)})$$
$$\Big(g(c_{(i\to 1)}) - g(c_{(i\to 0)})\Big)^T \Big(f(d_{(j\to\diamond)}) - f(d_{(j\to c_j)})\Big) = 0$$

□

Corollary 8. *Under the conditions of Theorem 7, if concept C_i and C_j have matched-representations, then*

$$g(c_{(i\to 1)}) - g(c_{(i\to 0)}) \perp g(c_{(j\to 1)}) - g(c_{(j\to 0)})$$

for any $c \in \mathscr{C}$.

Proof. Consider two binary vectors $c^{(0)}, c^{(1)} \in \mathscr{C}$ where $c_j^{(0)} = 0$ and $c_j^{(1)} = 1$ but they agree on other entries.

By Theorem 7,

$$g(c^{(0)}_{(i\to 1)}) - g(c^{(0)}_{(i\to 0)}) \perp f(d^{(0)}_{(j\to c_j)}) - f(d^{(0)}_{(j\to\diamond)})$$

for any $d^{(0)} \in \mathscr{D}_{c^{(0)}}$. Similar statements can be made for $c^{(1)}$ as well.

Note that $\mathscr{D}_{c^{(0)}} \cap \mathscr{D}_{c^{(1)}} \neq \emptyset$ by the positivity assumption. Let $d \in \mathscr{D}_{c^{(0)}} \cap \mathscr{D}_{c^{(1)}}$. Then,

$$g(c^{(0)}_{(i\to 1)}) - g(c^{(0)}_{(i\to 0)}) \perp f(d_{(j\to 0)}) - f(d_{(j\to\diamond)})$$
$$g(c^{(1)}_{(i\to 1)}) - g(c^{(1)}_{(i\to 0)}) \perp f(d_{(j\to 1)}) - f(d_{(j\to\diamond)})$$

By assumption, there exists a unit vector u_i, such that

$$g(c^{(0)}_{(i\to 1)}) - g(c^{(0)}_{(i\to 0)}) = \alpha^{(0)} u_i \quad g(c^{(1)}_{(i\to 1)}) - g(c^{(1)}_{(i\to 0)}) = \alpha^{(1)} u_i$$

for some $\alpha^{(0)}, \alpha^{(1)} > 0$. Therefore,

$$\Big\langle u_i, f(d_{(j\to 0)}) - f(d_{(j\to\diamond)}) \Big\rangle = \Big\langle u_i, f(d_{(j\to 1)}) - f(d_{(j\to\diamond)}) \Big\rangle = 0$$

Thus,

$$\Big\langle u_i, f(d_{(j\to 0)}) - f(d_{(j\to 1)}) \Big\rangle = 0$$

Because latent variable C_i has linaer and matched representations,

$$g(c_{(i\to 1)}) - g(c_{(i\to 0)}) \perp g(c_{(j\to 1)}) - g(c_{(j\to 0)})$$

for any $c \in \mathscr{C}$. □

E Simulated Experiments

In this section, we will provide additional details on our simulated experiments with the latent conditional model.

Additional details To let models learn conditional distributions, we train them to make predictions using cross-entropy loss. To turn the aforementioned generated binary vectors into a prediction task, we randomly generate binary masks μ for each vector in the batch. And if $\mu_i = 1$, the i-th entry of the vector is left untouched, and if $\mu_i = 0$, then it is set to -1 which is a numerical representation of the token $\diamond$. The model is trained to use masked vectors to predict the original vectors.

These sampled binary vectors and ternary vectors are mapped into one-hot encodings to avoid neural networks exploiting the inherent structures of these vectors. f and g are modeled as a linear function, which are essentially lookup tables. This construction is made without loss of generality.

Adam optimizer Although the theory is presented with gradient descent, the empirical result is actually robust to the choice of optimizers. We repeat the previous experiments on the complete set of conditionals with Adam optimizer [KB15] using a learning rate 0.001 and observe similar linear representation patterns in Table 2.

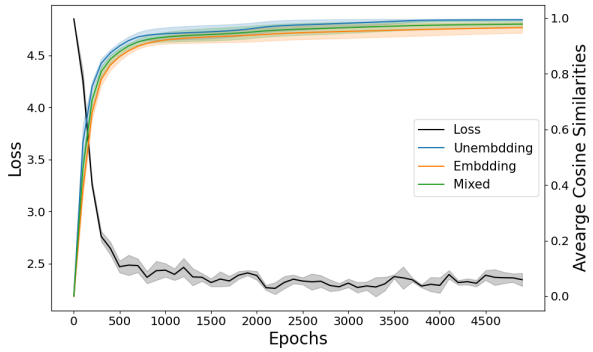
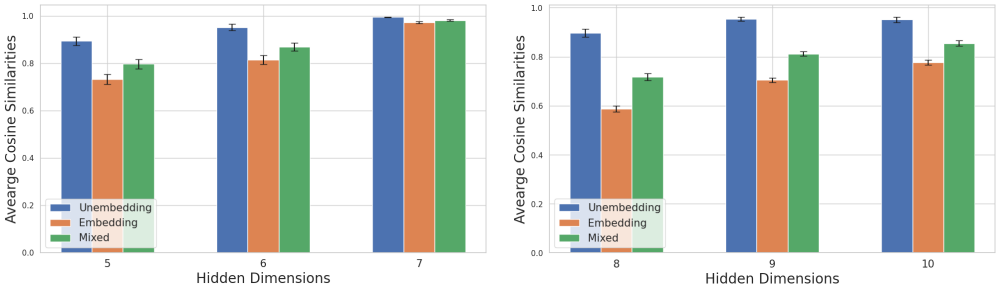


Figure 5: Loss and cosine similarities change as training progresses. The experiments are tested with 7 hidden variables over 10 runs.

Table 2: When the model is trained with Adam optimizer, the m latent variables are represented linearly, and the embedding and unembedding representations are matched. The table shows average cosine similarities among and between steering vectors of unembeddings and embeddings. Standard errors are over 100 runs for 3 variables and 4 variables, 50 runs for 5 variables, 20 runs for 6 variables and 10 runs for 7 variables.

m	Unembedding	Embedding	Unembedding and Embedding
3	0.910±0.015	0.923±0.013	0.926±0.012
4	0.972±0.005	0.959±0.005	0.965±0.005
5	0.985±0.004	0.970±0.004	0.975±0.004
6	0.996±0.001	0.977±0.002	0.983±0.001
7	0.966±0.012	0.918±0.016	0.929±0.015

Incomplete set of contexts ($\widehat{\mathscr{D}} \subset \mathscr{D}$) Both the size of $\mathscr{C}$ and $\mathscr{D}$ grow exponentially. To model large language models, not every conditional probability is necessarily trained. In fact, one does not need the complete set of contexts to get linearly encoded representations. To test this, we run experiments on incomplete subsets of contexts by randomly selecting a



(a) 7 Hidden Variables **(b)** 10 Hidden Variables

Figure 6: Average cosine similarities under different hidden dimensions show that reducing dimension dose not hurt linearity significantly.

few masks $\{\mu\}$. Table 3 shows that even with subsets of contexts, one can still get linearly encoded representations.

Table 3: When the model is trained to learn subsets of conditionals for 10 latent variables, the latent variables are still represented linearly and matched. The table shows average cosine similarities. Standard errors are over 10 runs.

Max Number of Masks	Unembedding	Embedding	Unembedding and Embedding
50	0.974±0.009	0.946±0.014	0.959±0.011
100	0.957±0.009	0.915±0.013	0.934±0.011

Incomplete set of concept vectors or violation of positivity ($\hat{\mathscr{C}} \subset \mathscr{C}$) In real-world language modeling, not every concept vector will be mapped to a token. We test this by allowing some concept vectors c to have zero probabilities (i.e., $p(c) = 0$). For the experiments, we first randomly select a subset $\hat{\mathscr{C}}$ of $\mathscr{C}$ and then use rejection sampling to collect data points. Table 4 shows that one can still get reasonable linearity for unembeddings. The embedding alignments drop, however. One possible explanation is due to a lack of training because the size of the problem grows exponentially. On the other hand, Section 3.2 suggests the connection between linearity and zero conditional probabilities. Because the positivity assumption is violated, the newly introduced zero conditional probabilities might also cause misalignment.

Table 4: When training with an incomplete set of concept vectors and contexts, unembedding representations are still encoded linearly. The table shows average cosine similarities. Standard errors are over 10 runs.

Number of Hidden Varibles	Unembedding	Embedding	Unembedding and Embedding
10	0.951±0.011	0.777±0.010	0.855±0.011
12	0.896±0.011	0.551±0.008	0.696±0.009

Change of dimensions Previous experiments set the representation dimension to be the same as the number of latent variables. In this set of experiments, we test how decreasing dimensions affects representations by rerunning experiments on 7 variables with the complete set of contexts and binary vectors as well as experiments with 10 variables with incomplete set of contexts and binary vectors. Figure 6 shows that although decreasing dimensions do make representations less aligned, the effect is not significant.